



תיעוד טכני

מערכת קשר קולי מבוססת דפדפן ברשת מקומית

מסמך העברה למפתח

שורות 226 — `server.py`
שורות 307 — `client.html`
שורות 375 — `manager.html`

גרסה 1.0 · ספטמבר 2026

1. סקירה

מערכת אינטרקום לרשת מקומית סגורה. שרת Python יחיד מגיש שני דפי HTML סטטיים ומנתב ביניהם אודיו גולמי מעל WebSocket. אין בסיס נתונים, אין תלויות חיצוניות פרט ל- `websockets`, ואין תהליך בנייה. בעמדות הקצה רץ דפדפן בלבד.

קובץ	אחריות
<code>server.py</code>	שרת HTTP סטטי (פורט 8000) + שרת WebSocket (פורט 8001), ניהול מצב גלובלי וניתוב פריימי אודיו
<code>client.html</code>	עמדת קצה: הצטרפות לצוות, לכידת מיקרופון, ניגון ומיקס של דוברים מרובים
<code>manager.html</code>	לוח בקרה: הפעלת צוותים, שידור כללי, הצטרפות לערוץ צוות, תצוגת מחוברים



כל האודיו עובר דרך השרת — טופולוגיית כוכב, ללא P2P. השרת אינו מפענח, מערבב או ממיר אודיו; הוא מעביר בייטים ומחליט למי.

2. הפרוטוקול

ערוץ WebSocket יחיד נושא שני סוגי פריימים. פריים טקסט הוא JSON של בקרה, פריים בינארי הוא אודיו. ההפרדה נעשית ב- `isinstance(message, bytes)` בשרת ובבדיקת `typeof event.data` בדפדפן.

2.1 הודעות בקרה

מהדפדפן לשרת

הודעה	הערות
<code>{type:"hello", role:"client", name, group}</code>	רישום לקוח. <code>group</code> חייב להימצא ברשימה, אחרת מוחזר <code>error</code>
<code>{type:"hello", role:"manager"}</code>	רישום מפקד
<code>{type:"broadcast", on}</code>	מפקד בלבד
<code>{type:"group", group, on}</code>	מפקד בלבד. מפעיל ערוץ פנים-צוותי
<code>{type:"join", group, on}</code>	מפקד בלבד. צוות אחד בכל רגע

מהשרת לדפדפן

הודעה	הערות
<code>{type:"groups", groups:[...]}</code>	נשלח מיד עם פתיחת החיבור, לפני ה- <code>hello</code>
<code>{type:"welcome", id, role}</code>	אישור רישום
<code>{type:"state", status}</code>	ללקוח: <code>BROADCAST</code> / <code>ACTIVE</code> / <code>STANDBY</code>
<code>{type:"state", broadcast, active, joined, members}</code>	למפקד: תמונת מצב מלאה
<code>{type:"error", message}</code>	כשלון רישום

שידור מצב

`push_state()` (שורה 102) נקרא אחרי כל שינוי מצב ואחרי כל התחברות או ניתוק, ומשדר לכל המחוברים במקביל דרך `asyncio.gather`. אין דלתאות — תמיד נשלחת תמונה מלאה. זה מייתר סנכרון עדין בצד הלקוח.

2.2 פריים אודיו

הדפדפן שולח PCM גולמי ללא כותרת. השרת מוסיף מזהה שולח לפני ההעברה, כדי שהמקבל יוכל להפריד בין דוברים ולמקסס אותם:

<code>peer id</code> uint32 little-endian · 4B	Float32 PCM mono 1024 samples · 4096B
---	--

גודל הפריים היוצא מהשרת הוא 4100 בייט. היישור ל-4 בייטים מכון: הוא מאפשר `new Float32Array(buffer)` על התוצאה של `slice(4)` ללא העתקה נוספת.

פרמטר	ערך	נגזרת
קצב דגימה	48000 Hz	נדרש מ- <code>AudioContext</code> בשני הדפים
גודל מנה	1024 דגימות	21.3ms · 46.9 פריימים לשנייה
רוחב פס לזרם	1.54 Mbps	192 KB/s, ללא הפסקה

3. מודל המצב

כל המצב יושב במופע יחיד של `State` (שורה 30) בזיכרון התהליך. אין התמדה — הפעלה מחדש מאפסת הכול.

```
broadcast      : bool      # שידור כללי פעיל
active_groups  : set[str]   # צוותים שערוך הפנים שלהם פתוח
joined_group   : str | None # הצוות שהמפקד מחובר אליו
peers          : dict[int, Peer]
```

גזירת המצב של הלקוח

`status_for()` (שורה 70) — הלקוח אינו מחזיק מצב משלו, הוא מקבל אותו מוכן:

תנאי	מצב	משמעות
<code>broadcast</code>	BROADCAST	קליטה בלבד, מיקרופון מושתק
<code>group in active_groups</code> או <code>joined_group == group</code>	ACTIVE	משדר ברציפות
אחרת	STANDBY	לא משדר, לא קולט

מצב גלובלי, לא לכל מפקד

יותר ממפקד אחד יכול להתחבר, אך כולם חולקים את אותו `State` ורואים אותו לוח בקרה. פעולה של מפקד אחד משפיעה על השאר מיד. אם נדרשים מפקדים עצמאיים, המצב צריך לעבור להיות פר-מפקד — זה שינוי מבני ולא פרמטר.

4. מטריצת ניתוב האודיו

זהו לב המערכת: `audio_targets(sender)` (שורה 78). הפונקציה מקבלת את השולח ומחזירה את רשימת המקבלים. כל פריים בינארי עובר דרכה.

מצב	שולח	מקבלים
broadcast = True	מפקד	כל המחוברים פרט לשולח
	לקוח	רשימה ריקה — הפריים נזרק
רגיל	מפקד	לקוחות <code>joined_group</code> בלבד; ריק אם לא הצטרף
	לקוח	חברי הצוות (אם הצוות ב- <code>active_groups</code>) + המפקדים (אם <code>joined_group</code>) הוא הצוות (שלו)

אכיפה כפולה, בכוונה

גם הדפדפן מגביל שליחה (`status == "ACTIVE"` בלקוח, `transmitting()` במפקד) וגם השרת מסנן קליטה. הגבלת הדפדפן חוסכת רוחב פס; הסינון בשרת הוא זה שקובע. לקוח ששולח כשאסור לו — הפריים שלו פשוט נזרק.

שים לב לתוצאה הריבועית: בצוות פעיל בן K חברים, כל פריים נכנס משוכפל ל- $K-1$ יוצאים. תעבורת השרת גדלה כ- $K(K-1)$, ולא ליניארית.

5. server.py

מודל ההרצה

שני מנגנוני מקביליות חיים זה לצד זה:

- **לולאת asyncio** — כל תעבורת ה-WebSocket. משימה אחת לכל חיבור, נוצרת על ידי `serve()`.
- **thread נפרד** — `ThreadingHTTPServer` (שורה 203) מגיש את קובצי ה-HTML מ-`BASE_DIR`. `daemon thread`. נסגר עם התהליך.

שני העולמות אינם חולקים מצב, ולכן אין צורך בנעילות. `QuietHandler` קיים רק כדי להשתיק את לוג הגישות.

מחזור חיי חיבור

`handler()` (שורה 166):

```
1. state.peers, רישום ב id עם Peer יצירת
2. {type:"groups"} שליחת # עוד לפני זיהוי
3. לולאת הודעות:
   bytes -> struct.pack("<I", id) + data -> audio_targets()
   str -> handle_control()
4. finally: - ואם לא נותרו מפקדים, הסרה מ-peers
   joined_group = None, broadcast = False
5. push_state()
```

איפוס המצב ביציאת המפקד האחרון הוא הגנה מכוונת: בלעדיו נפילת דפדפן המפקד הייתה משאירה את כל הלקוחות ב-`ACTIVE` עם מיקרופון פתוח ללא הגבלת זמן.

לחץ חוזר (backpressure)

העברת הפריים היא `await asyncio.gather(*[t.ws.send(frame) for t in targets])`. אם מקבל אחד איטי ומאגר השליחה שלו מתמלא, ה-`await` מעכב את משימת השולח — כלומר האודיו שלו נעצר לכולם, לא רק למקבל התקוע. תיקון אפשרי: תור פר-מקבל עם השלכת פריימים ישנים במקום המתנה.

תאימות גרסאות

הייבוא מנסה `websockets.asyncio.server` ונופל חזרה ל-`websockets.server`. חתימת `handler(ws, path=None)` מכסה גם את ה-API הישן שהעביר `path` וגם את החדש שאינו מעביר.

קובץ הצוותים

— `groups.txt` — שם צוות בכל שורה, נוצר עם ברירת מחדל אם חסר. נקרא **פעם אחת** ב-`main()`. שינוי דורש הפעלה מחדש של השרת.

6. צינור האודיו בדפדפן

זהה בשני הדפים. שני `AudioWorkletProcessor` נטענים כ-Blob URL, כדי לשמור על קובץ בודד ללא קבצי לוואי.

`getUserMedia → MediaStreamSource → recorder worklet → ws.send()`

▼
`ws.onmessage → split(4) → player worklet (mix) → destination`

recorder

ה-worklet מקבל מנות של 128 דגימות מהמונע וצובר אותן ל-1024 לפני `postMessage`. ה-`ArrayBuffer` מועבר ב-transfer list ולא מועתק. הצומת מחובר ל-`GainNode` עם `gain = 0` אל היעד, כי צומת ללא חיבור יציאה לא מובטח שיעובד.

player

מחזיק `Map` של תור מנות לכל מזהה שולח. בכל `process()` נשלפות 128 דגימות מכל תור והן מסוכמות לתוך פס יציאה יחיד, עם `clamp` ל- ± 1 . זהו המקבילה למיקסר של הגרסה המקורית ב-Python. כל תור מוגבל ל-**12288 דגימות** (כ-256ms). בחריגה מזרקות המנות הישנות. זו ההגנה היחידה מפני הצטברות השהיה — אין jitter buffer אדפטיבי ואין תיקון סחיפת שעונים.

מדוע אין אובדן מנות

WebSocket רץ מעל TCP, ולכן אין אובדן — אבל יש head-of-line blocking: מנה שנתקעת מעכבת את כל שאחריה. במימוש UDP המקורי התנהגות זו הייתה הפוכה. בפועל, ברשת מקומית תקינה ההבדל אינו מורגש.

התחברות מחדש

`ws.onclose` מתזמן `connectSocket()` מחדש כל 2 שניות, ללא הגבלה. ה-`AudioContext` וההרשאה למיקרופון נוצרים פעם אחת ושורדים ניתוקים — רק ה-socket נבנה מחדש.

7. קבועים

קבוע	ערך	מיקום
HTTP_PORT	8000	server.py:17
WS_PORT	8001	server.py:18 · משוכפל בקשיח ב-WS_URL בשני דפי ה-HTML
SAMPLE_RATE	48000	שני דפי ה-HTML · חייב להיות זהה בכולם
CHUNK	1024	שני דפי ה-HTML · מוטבע בקוד ה-worklet דרך template literal
תקרת תור	12288	קוד ה-worklet player

שכפול קבועים

מספר הפורט 8001 מופיע בשלושה מקומות. שינוי דורש עדכון בכל השלושה. חלופה נקייה: לשרת את WS_PORT מהשרת כנקודת קצה קטנה, או להשתמש באותו פורט ל-HTTP ול-WebSocket דרך process_request.

8. מגבלות ידועות

נושא	פירוט
אין אימות	כל מי שמגיע לפורט יכול להתחבר, לבחור שם כרצונו ולקבל תפקיד manager. מתאים לרשת סגורה בלבד.
אין הצפנה	ws:// ו- http:// בלבד. תעבורה גלויה.
הקשר מאובטח	דפדפנים חוסמים getUserMedia מחוץ ל-localhost ב-HTTP. נדרשת הגדרת דגל בכל עמדה, או מעבר ל-HTTPS.
רוחב פס	Float32 לא דחוס — 1.54 Mbps לזרם. ריבועי בגודל הצוות.
קצב דגימה	48000 מבוקש ולא מנוהל משא ומתן. דפדפן שלא יכבד אותו יגרום לעיוות גובה צליל.
אין VAD	המיקרופון משדר גם בשקט מוחלט.
שקיפות ניטור	אין לוג, אין מדדים, אין ניטור. לאבחון — הדפסות ידניות.

9. נקודות הרחבה

שינוי	היקף	השפעה
Int16 במקום Float32	דפדפן בלבד	חצי מרוחב הפס. השרת אינו מושפע — הוא מעביר בייטים אטומים
הורדה ל-16kHz	דפדפן בלבד	פי 3 נוסף. דורש דגימה מחדש (resampling) בצד השולח
Push-to-talk / VAD	דפדפן בלבד	החיסכון הגדול ביותר — מבטל את התעבורה בזמן שקט
אימות	שדה ב- <code>hello</code>	אימות לפני הרישום ב- <code>peers</code> ; השם והצוות נלקחים מהאסימון ולא מהטופס
מצב widget	<code>client.html</code>	גרסה מצומצמת ב- <code>iframe</code> , זהות מגיעה ב- <code>postMessage</code> . דורש <code>allow="microphone"</code> במארח
TLS	שרת + דפים	<code>ssl_context</code> ב- <code>serve()</code> , מעבר ל- <code>wss://</code> , ותעודה מהימנה בעמדות

10. הרצה מקומית

```
pip install websockets
python server.py

# http://<server-ip>:8000/client.html
# http://<server-ip>:8000/manager.html
```

הפעלה ראשונה יוצרת `groups.txt` עם שלושה צוותים לדוגמה. יש לפתוח את פורטים 8000 ו-8001 בחומת האש של מחשב השרת.

לבדיקה מהירה על מחשב אחד: פתח שתי לשוניות `localhost` — אחת מפקד ואחת לקוח. `localhost` נחשב הקשר מאובטח ולכן המיקרופון עובד ללא הגדרות נוספות.